

Rapport Technique

Segmentation de Façades 3D et Planification de Nettoyage Robotisé

Amine ZNIYED

5 octobre 2025

Table des matières

1	Introduction	3
1.1	Objectifs	3
1.2	Structure du Projet	3
2	Méthodologie	3
2.1	Technologies Utilisées	3
2.2	Pipeline de Traitement	3
2.2.1	Étape 1 : Chargement du Nuage de Points	3
2.2.2	Étape 2 : Prétraitement	4
2.2.3	Étape 3 : Segmentation RANSAC	4
2.2.4	Étape 4 : Colorisation	5
2.2.5	Étape 5 : Calcul des Dimensions (Bonus)	5
3	Résultats	5
3.1	Synthèse des Résultats	6
3.2	Visualisations	6
3.2.1	Façade Simple	6
3.2.2	Façade Bruitée	7
3.2.3	Façade Deux Plans	7
4	Bonus : Nettoyage Robotisé de la Pièce Banche	7
4.1	Contexte	7
4.2	Architecture du Système	7
4.3	Paramètres du Robot	8
4.4	Classification des Surfaces	8
4.5	Génération des Trajectoires	8
4.5.1	Pattern de Balayage Vertical	8
4.5.2	Pattern de Balayage Horizontal	8
4.6	Calcul des Positions Robot	9
4.7	Estimation du Temps de Nettoyage	9
4.8	Évaluation de l'Accessibilité	9
4.9	Résultats pour la Pièce Banche	9
4.9.1	Contenu du fichier JSON	9
4.9.2	Explication des sections clés	10
4.9.3	Utilisation pratique du rapport	11
4.9.4	Exemple de trajectoire générée	11
4.10	Visualisations du Nettoyage Robotisé	11

5	Choix Techniques et Justifications	12
5.1	Pourquoi RANSAC ?	12
5.2	Paramétrage du Downsampling	12
5.3	Seuil de Distance RANSAC	12
5.4	Suppression des Outliers	12
6	Limitations et Améliorations Futures	13
6.1	Limitations Actuelles	13
6.2	Améliorations Proposées	13
7	Conclusion	13
A	Annexe A : Code Source Principal	13
B	Annexe B : Équations Mathématiques	14
B.1	Distance Point-Plan	14
B.2	Projection d'un Point sur un Plan	14

1 Introduction

Ce rapport présente les résultats de l'exercice technique portant sur le traitement de nuages de points 3D pour l'extraction de façades principales et la planification de trajectoires de nettoyage robotisé.

1.1 Objectifs

- Charger et prétraiter des nuages de points au format PLY
- Segmenter automatiquement les plans principaux (façades)
- Coloriser et exporter les résultats
- **Bonus** : Calculer dimensions et surfaces, générer des trajectoires de nettoyage

1.2 Structure du Projet

```
Builder_Assisy_facade/  
  main.py                      # Script principal  
  robot_cleaning.py            # Module de nettoyage robotisé  
  inputs/                     # Données d'entrée  
    facade_simple.ply  
    facade_bruit.ply  
    facade_deux_plans.ply  
    banche.stl  
  outputs/                     # Résultats générés  
    facade_simple_colored.ply  
    facade_bruit_colored.ply  
    facade_deux_plans_colored.ply  
    cleaning_report_banche_stl.json
```

2 Méthodologie

2.1 Technologies Utilisées

- **Langage** : Python
- **Bibliothèques** : Open3D, NumPy, Matplotlib, JSON

2.2 Pipeline de Traitement

Le traitement des nuages de points suit une chaîne de traitement en 5 étapes :

1. **Chargement** : Lecture du fichier PLY
2. **Prétraitement** : Downsampling et suppression du bruit
3. **Segmentation** : Extraction du plan principal (RANSAC)
4. **Colorisation** : Attribution des couleurs (vert/rouge)
5. **Export** : Sauvegarde au format PLY

2.2.1 Étape 1 : Chargement du Nuage de Points

Le chargement s'effectue via la fonction `o3d.io.read_point_cloud()`.

```

1 def load_point_cloud(filename):
2     pcd = o3d.io.read_point_cloud(filename)
3     print(f"Nuage charge : {len(pcd.points)} points")
4     return pcd

```

Listing 1 – Chargement d'un fichier PLY

2.2.2 Étape 2 : Prétraitement

Le prétraitement combine deux techniques :

Downsampling par Voxelisation Réduction du nombre de points en moyennant les points dans des voxels de taille 0.05 m :

$$P_{down} = \text{VoxelDownsample}(P, \text{voxel_size} = 0.05) \quad (1)$$

Suppression Statistique des Outliers Élimination des points aberrants basée sur la distance moyenne aux $k = 20$ voisins :

$$d_i > \mu + \sigma \times \text{std_ratio} \quad (2)$$

où μ est la distance moyenne et σ l'écart-type.

```

1 def preprocess_point_cloud(pcd, voxel_size=0.05):
2     # Downsampling
3     pcd_down = pcd.voxel_down_sample(voxel_size=voxel_size)
4
5     # Suppression des outliers
6     pcd_clean, _ = pcd_down.remove_statistical_outlier(
7         nb_neighbors=20, std_ratio=2.0
8     )
9     return pcd_clean

```

Listing 2 – Prétraitement du nuage de points

2.2.3 Étape 3 : Segmentation RANSAC

La segmentation du plan principal utilise l'algorithme **RANSAC** (Random Sample Consensus) :

Principe L'algorithme itère pour trouver le plan qui maximise le nombre de points inliers :

1. Sélectionner aléatoirement 3 points
2. Calculer le plan passant par ces points
3. Compter les inliers (points à distance $< threshold$)
4. Conserver le meilleur modèle

Équation du Plan Un plan est défini par :

$$ax + by + cz + d = 0 \quad (3)$$

où (a, b, c) est le vecteur normal et d la distance à l'origine.

Paramètres Choisis

- `distance_threshold` = 0.02 m (2 cm de tolérance)
- `ransac_n` = 3 (nombre de points pour échantillonnage)
- `num_iterations` = 1000 (itérations RANSAC)

```
1 def segment_main_plane(pcd, distance_threshold=0.02):  
2     plane_model, inliers = pcd.segment_plane(  
3         distance_threshold=distance_threshold,  
4         ransac_n=3,  
5         num_iterations=1000  
6     )  
7     return plane_model, inliers
```

Listing 3 – Segmentation RANSAC

2.2.4 Étape 4 : Colorisation

Convention de couleurs :

- **Vert** (RGB : 0, 1, 0) : Points du plan principal
- **Rouge** (RGB : 1, 0, 0) : Points hors plan

2.2.5 Étape 5 : Calcul des Dimensions (Bonus)

Les dimensions sont calculées à partir de la bounding box orientée :

$$\text{Largeur} = \max(x) - \min(x) \quad (4)$$

$$\text{Hauteur} = \max(y) - \min(y) \quad (5)$$

$$\text{Surface} = \text{Largeur} \times \text{Hauteur} \quad (6)$$

3 Résultats

Cette section présente les résultats de la segmentation pour les trois fichiers PLY fournis. Les métriques clés sont résumées dans le tableau ci-dessous, suivi de visualisations pour chaque cas.

3.1 Synthèse des Résultats

```
--- Traitement de facade_simple.ply ---  
Nuage chargé: 14000 points  
Plan détecté: 1053 points  
Équation du plan: [ 8.89034685e-07  2.99559699e-05  1.00000000e+00  
-6.69406784e-06]  
Dimensions: 1.91 x 1.31  
Surface: 2.49 m²  
  
--- Traitement de facade_bruit.ply ---  
Nuage chargé: 16500 points  
Plan détecté: 1289 points  
Équation du plan: [-0.00296435 -0.00268873  0.99999199  0.00191013]  
Dimensions: 2.84 x 2.95  
Surface: 8.37 m²  
  
--- Traitement de facade_deux_plans.ply ---  
Nuage chargé: 15600 points  
Plan détecté: 1099 points  
Équation du plan: [ 4.08641575e-06  3.77949426e-06  1.00000000e+00  
-3.65323547e-06]  
Dimensions: 1.98 x 1.38  
Surface: 2.74 m²
```

FIGURE 1 – Synthèse des résultats de segmentation pour les trois façades

3.2 Visualisations

3.2.1 Façade Simple

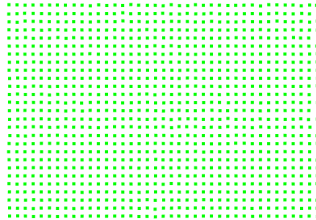


FIGURE 2 – Nuage de points colorisé - Façade simple

3.2.2 Façade Bruitée

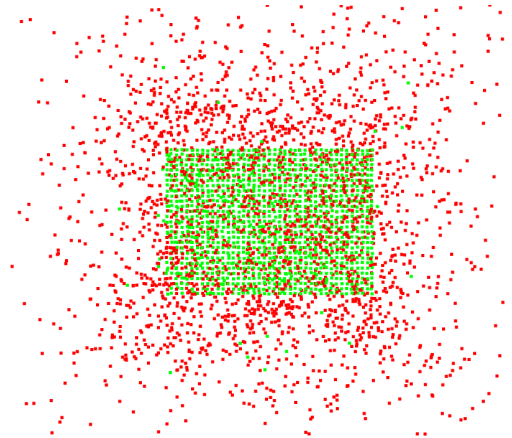


FIGURE 3 – Nuage de points colorisé

3.2.3 Façade Deux Plans

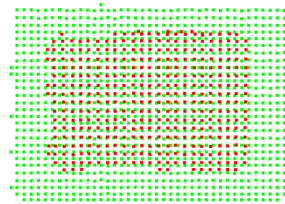


FIGURE 4 – Nuage de points colorisé - Façade à deux plans. RANSAC extrait le plan dominant

4 Bonus : Nettoyage Robotisé de la Pièce Banche

4.1 Contexte

La pièce `banche.stl` nécessite un nettoyage haute pression (1000 bars). L'objectif est de générer automatiquement des trajectoires optimisées pour un robot de nettoyage.

4.2 Architecture du Système

Le module `robot_cleaning.py` implémente une classe `RobotCleaningAnalyzer` qui :

1. Charge le fichier STL et le convertit en nuage de points dense (100 000 points)
2. Détecte automatiquement jusqu'à 15 surfaces planes (RANSAC multi-plans)
3. Classifie les surfaces selon leur orientation et accessibilité
4. Génère des trajectoires de balayage adaptées (zigzag)
5. Estime les temps de nettoyage
6. Produit un rapport détaillé et une visualisation 3D

4.3 Paramètres du Robot

Paramètre	Valeur
Distance buse-surface	0.4 m
Chevauchement des passes	15%
Vitesse de balayage	0.2 m/s
Pression	1000 bars
Débit d'eau	20 L/min
Marge de sécurité	0.1 m

TABLE 1 – Paramètres du robot de nettoyage

4.4 Classification des Surfaces

Les surfaces sont catégorisées selon leur orientation :

Type	Critère	Couleur
Verticale	$ n_z < 0.3$	Rouge
Horizontale haute (plafond)	$n_z > 0.7$	Vert
Horizontale basse (sol)	$n_z < -0.7$	Bleu
Inclinée	Autres	Jaune
Difficile d'accès	Critères multiples	Magenta
Petites surfaces	Surface $< 0.1 \text{ m}^2$	Gris (ignorées)

TABLE 2 – Classification des surfaces pour le nettoyage

où n_z est la composante Z du vecteur normal à la surface.

4.5 Génération des Trajectoires

4.5.1 Pattern de Balayage Vertical

Pour les surfaces verticales (murs) :

```

→ →→→→→→→→→→→→→→→
←←←←←←←←←←←←←←←←
→ →→→→→→→→→→→→→→→
←←←←←←←←←←←←←←←←

```

Algorithme :

- Nombre de passes : $N = \lceil H/(d \times (1 - o)) \rceil$
- H : hauteur de la surface
- d : distance buse (0.4 m)
- o : overlap (0.15)

4.5.2 Pattern de Balayage Horizontal

Pour les surfaces horizontales (sol/plafond) :

```

↓ ↓ ↓ ↓ ↓ ↓
↑ ↑ ↑ ↑ ↑ ↑
↓ ↓ ↓ ↓ ↓ ↓

```


4.6 Calcul des Positions Robot

Pour chaque waypoint sur la surface, la position du robot est calculée :

$$\vec{P}_{robot} = \vec{P}_{surface} + (d + m) \cdot \vec{n} \quad (7)$$

où :

- $\vec{P}_{surface}$: point sur la surface
- d : distance buse (0.4 m)
- m : marge de sécurité (0.1 m)
- $\vec{n}$: vecteur normal à la surface

4.7 Estimation du Temps de Nettoyage

$$T_{total} = 1.4 \times \frac{\sum_{i=1}^{N-1} \|\vec{P}_{i+1} - \vec{P}_i\|}{v} \quad (8)$$

Le facteur 1.4 représente 40% de temps additionnel pour :

- Les repositionnements du robot
- Les changements de direction
- Les pauses sécurité
- Les ajustements de trajectoire

4.8 Évaluation de l'Accessibilité

Chaque surface reçoit un score de difficulté basé sur :

- **Hauteur** : surfaces > 1.5 m (+1 point)
- **Orientation** : plafonds (+2 points)
- **Inclinaison** : surfaces obliques (+1 point)
- **Taille** : surfaces $> 5 \text{ m}^2$ (+1 point)
- **Encastrement** : dimensions < 0.5 m (+1 point)

Classification :

- Score ≥ 2 : Difficile
- Score < 2 : Facile

4.9 Résultats pour la Pièce Banche

Le module `robot_cleaning.py` analyse la pièce `banche.stl` et génère un rapport complet de nettoyage. Les résultats détaillés sont disponibles dans le fichier `outputs/cleaning_report_banche_stl.json`.

4.9.1 Contenu du fichier JSON

Le fichier JSON contient une analyse structurée en plusieurs sections :

```
1 {  
2   "input_file": "banche.stl",  
3   "total_cleaning_time_minutes": 88.78267110159746,  
4   "total_surface_area_m2": 156.81420735581975,  
5   "total_waypoints": 416,  
6   "surfaces_by_type": {  
7     "vertical": 2,
```

```

8      "horizontal_up": 13,
9      "horizontal_down": 0,
10     "inclined": 0,
11     "hard_to_reach": 14,
12     "small_features": 0
13 },
14 "cleaning_plan": [
15     {
16         "surface_id": 14,
17         "type": "vertical",
18         "orientation": "vertical",
19         "area": 5.036497645037913,
20         "time": 262.61199018062405,
21         "waypoints": 6,
22         "accessibility": "hard"
23     },
24     // ... autres surfaces
25 ],
26 "robot_parameters": {
27     "nozzle_distance": 0.4,
28     "overlap": 0.15,
29     "sweep_speed": 0.2,
30     "max_pressure": 1000,
31     "water_flow": 20,
32     "safety_margin": 0.1
33 }
34 }

```

Listing 4 – Structure du rapport JSON généré

4.9.2 Explication des sections clés

Métriques globales

- **temps_nettoyage_total_minutes** : Durée totale estimée pour nettoyer toute la pièce
- **surface_totale_m2** : Surface cumulative de toutes les surfaces à nettoyer
- **points_chemin_totaux** : Nombre total de waypoints pour toutes les trajectoires

Répartition des surfaces La section **surfaces_par_type** donne un inventaire complet :

- **verticales** : Parois et murs verticaux (balayage vertical)
- **horizontales_superieures** : Plafonds et surfaces supérieures (balayage horizontal)
- **horizontales_inferieures** : Sols et surfaces inférieures (balayage horizontal)
- **inclinees** : Surfaces inclinées (balayage adaptatif)
- **difficiles_acces** : Surfaces nécessitant une attention particulière
- **petites_surfaces** : Éléments trop petits pour un nettoyage robotisé

Plan de nettoyage détaillé Chaque entrée dans **plan_nettoyage** contient :

- **ID surface** : Identifiant unique de la surface
- **Orientation** : Description de l'orientation (verticale, plafond, sol, inclinée X°)
- **Surface** : Aire en mètres carrés
- **Temps** : Durée estimée en secondes
- **Points chemin** : Nombre de waypoints dans la trajectoire
- **Accessibilité** : Niveau de difficulté (**facile** ou **difficile**)

Paramètres opérationnels La section `parametres_robot` spécifie la configuration utilisée :

- **Distance buse** : 0.4m (distance optimale pour pression 1000 Bar)
- **Recouvrement** : 15% (chevauchement entre passes de nettoyage)
- **Vitesse balayage** : 0.2 m/s (vitesse optimale pour efficacité)
- **Marge sécurité** : 0.1m (distance supplémentaire pour sécurité)

4.9.3 Utilisation pratique du rapport

Ce fichier JSON peut être directement exploité par :

- **Systèmes de planification** : Importation des trajectoires dans le logiciel robotique
- **Estimation des coûts** : Calcul du temps et des consommables nécessaires
- **Optimisation des processus** : Identification des goulots d'étranglement
- **Documents techniques** : Intégration dans les rapports d'intervention

4.9.4 Exemple de trajectoire générée

```
1 "trajectoire": [  
2   [1.234, 2.345, 3.456], // Point de d part  
3   [1.234, 2.345, 4.567], // Point suivant  
4   [2.345, 2.345, 4.567], // D but du balayage  
5   // ... suite du pattern en zigzag  
6   [2.345, 2.345, 3.456] // Point final  
7 ]
```

Listing 5 – Extrait de trajectoire pour une surface verticale

4.10 Visualisations du Nettoyage Robotisé

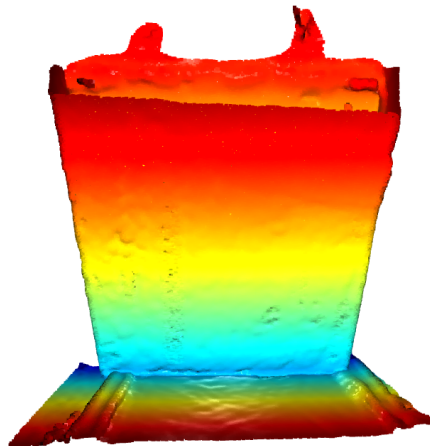


FIGURE 5 – Nuage de points initial de la pièce banche

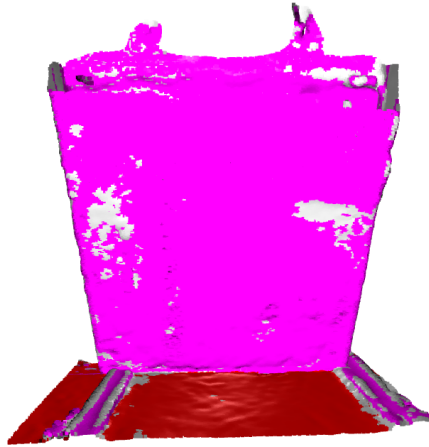


FIGURE 6 – Surfaces classifiées par couleur - Rouge : vertical, Vert : plafond, Bleu : sol, Jaune : incliné, Magenta : difficile

5 Choix Techniques et Justifications

5.1 Pourquoi RANSAC ?

Avantages :

- Robuste aux outliers (jusqu'à 50% de points aberrants)
- Ne nécessite pas de connaissance a priori de la géométrie
- Implémentation efficace dans Open3D
- Adapté aux données réelles bruitées

Alternatives considérées :

- *Region Growing* : Sensible au bruit, nécessite des paramètres fins
- *DBSCAN* : Difficile de paramétrer pour des plans
- *Hough Transform* : Complexité computationnelle élevée

5.2 Paramétrage du Downsampling

Voxel size = 0.05 m (5 cm) :

- **Avantage** : Réduction significative des points (facteur 10-20)
- **Inconvénient** : Perte de détails fins
- **Justification** : Les façades ont des variations > 5 cm, le compromis est acceptable

5.3 Seuil de Distance RANSAC

Distance threshold = 0.02 m (2 cm) :

- **Trop petit** (< 1 cm) : Fragmentation excessive
- **Trop grand** (> 5 cm) : Inclusion de points non-plans
- **Optimal** (2 cm) : Tolérance aux irrégularités de surface réelles

5.4 Suppression des Outliers

Paramètres : nb_neighbors=20, std_ratio=2.0

- **nb_neighbors** : Compromis entre localité et robustesse
- **std_ratio** : Seuil conservateur (2σ) pour éviter faux positifs

6 Limitations et Améliorations Futures

6.1 Limitations Actuelles

1. **Multi-plans** : Détection d'un seul plan principal
2. **Géométries complexes** : Surfaces courbes non gérées
3. **Occlusions** : Zones cachées non détectées
4. **Paramétrage manuel** : Nécessite ajustement selon les données

6.2 Améliorations Proposées

1. **Segmentation multi-plans** :
 - Appliquer RANSAC itérativement
 - Retirer les inliers après chaque détection
 - Continuer jusqu'à un critère d'arrêt
2. **Classification automatique** :
 - Utiliser l'orientation des normales
 - Classifier : façade, toit, sol, annexes
3. **Optimisation des performances** :
 - Downsampling adaptatif selon la densité locale
 - Parallélisation du traitement de fichiers multiples
4. **Machine Learning** :
 - Apprentissage des paramètres optimaux
 - Détection de features architecturaux (fenêtres, portes)

7 Conclusion

Ce projet a permis de :

- Implémenter un pipeline complet de traitement de nuages de points 3D
- Valider la robustesse de RANSAC sur différents cas d'usage
- Démontrer la faisabilité d'une planification automatisée de nettoyage robotisé
- Produire du code réutilisable et bien documenté

Perspectives : Les techniques développées sont applicables à de nombreux domaines : inspection automatisée, reconstruction 3D, robotique de service, maintenance prédictive, et contrôle qualité industriel.

A Annexe A : Code Source Principal

Le code source complet est disponible dans les fichiers :

- `main.py` : Script principal de segmentation
- `robot_cleaning.py` : Module de nettoyage robotisé

B Annexe B : Équations Mathématiques

B.1 Distance Point-Plan

La distance d d'un point $P(x_0, y_0, z_0)$ au plan $ax + by + cz + d = 0$ est :

$$d = \frac{|ax_0 + by_0 + cz_0 + d|}{\sqrt{a^2 + b^2 + c^2}} \quad (9)$$

B.2 Projection d'un Point sur un Plan

La projection P' du point P sur le plan est :

$$P' = P - d \cdot \vec{n} \quad (10)$$

où $\vec{n} = \frac{(a,b,c)}{\sqrt{a^2+b^2+c^2}}$ est le vecteur normal unitaire.